Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Course Project · Fall 2026

Team leader: Max Lin **Members:** Max Lin (individual project)

Code: GitHub repository (link submitted on Canvas); every number in this report is produced by `run_all.sh` and stored in `results/`.

Summary. We collected 9,636 responses written by 12 LLMs from 6 families (GPT, Claude, Llama, Qwen, Mistral, Google) to the *same* 803 user prompts, and trained a convolutional network (CNN) and a bidirectional LSTM (RNN) to predict the family. From the response alone the CNN reaches **56.2%** accuracy and the BiLSTM **53.0%** on unseen prompts (chance = 16.7%). Because every prompt was answered by every family, the prompt alone carries no information (input-only = chance), and adding it to the response does not help. A simulated data-collection bias shows how input-only classifiers can nevertheless look accurate on real-world datasets (RQ2).

1. Problem Definition and Dataset Curation

1.1 Task

Given a text produced by a chat LLM (and optionally the user prompt that produced it), predict which **LLM family** wrote it. Each example is a triple *(LLM_name, LLM_input, LLM_output)*. We classify at the family level (6 classes) rather than the exact model because the assignment asks about LLM families, and because two versions from one vendor (e.g. GPT-4-Turbo and GPT-4o) are trained with similar data and post-training recipes, so the family is the natural unit of a "fingerprint". We also keep the exact model name, which we use for a per-model analysis.

1.2 Data source and collection

Generating thousands of responses ourselves through paid APIs was not feasible, so we obtained publicly released outputs from **AlpacaEval** (Li et al., 2023; github.com/tatsu-lab/alpaca_eval, Apache-2.0 license). AlpacaEval is a leaderboard that stores, for every evaluated model, its raw answer to the same 805 instructions (`results/<model>/model_outputs.json`). The instructions come from five public instruction sets (Self-Instruct, OpenAssistant, Koala, Anthropic helpful-base, Vicuna) and cover advice, factual questions, creative writing, coding and math. The data contains no private or personal information; prompts were written by researchers or crowd-workers for public benchmarks.

From the ~230 entries on the leaderboard we kept only **official vendor models** (no community fine-tunes such as Vicuna or Zephyr, whose "family" would be ambiguous) and chose two models per family so that every family is represented by more than one model version (Table 1). This matters: if each family were a single model, the classifier could memorize one model's quirks instead of a family fingerprint.

Table 1. Dataset composition. Every prompt is answered by all 12 models, so classes are perfectly balanced.

Family	Models (AlpacaEval name)	Responses	Mean words	Median words
Claude	claude-3-5-sonnet-20240620, claude-3-opus-20240229	1,606	222	207
GPT	gpt-4o-2024-05-13, gpt4_1106_preview	1,606	305	319
Google	gemini-pro, gemma-7b-it	1,606	203	200
Llama	Meta-Llama-3-70B-Instruct, llama-2-70b-chat-hf	1,606	296	298
Mistral	Mixtral-8x7B-Instruct-v0.1, mistral-large-2402	1,606	229	218
Qwen	Qwen1.5-72B-Chat, Qwen2-72B-Instruct	1,606	252	252
Total	12 models	9,636		

1.3 Cleaning, splitting and annotation

(1) We removed empty responses (1 from gemini-pro) and then kept only prompts that *all 12* models answered, leaving **803 prompts × 12 = 9,636 examples** (1,606 per family). Prompts are short (mean 29 words, median 18); responses average 200–300 words. (2) We split **by prompt**, not by row: 70/15/15% of the prompts (562/120/121) go to train/validation/test together with all 12 of their answers (6,744/1,440/1,452 rows). A random row split would put GPT's answer to a prompt in training and Claude's answer to the same prompt in test, letting the model use topic words; the prompt split guarantees that every test prompt is new. (3) We also tag every prompt with a task type using keyword rules (`data/build_dataset.py`): **technical** (code, math, data formats; 85 prompts), **writing** (stories, poems, emails, rewriting; 148) and **general QA** (everything else: explanations, advice, facts; 570). We manually checked a sample of the tags; they are imperfect (a few borderline prompts land in the wrong bucket) but adequate for a coarse cross-task test.

Why this design. Because all families answer the same prompts, any difference the classifier picks up must come from *how* the model writes, not from *what* it was asked. This controlled design is what makes RQ2 interpretable (Section 3.2). The trade-off is that responses come from one benchmark and one time period (2023–2024 model versions), with each vendor's default API settings.

2. CNN / RNN Implementation and Training

2.1 Text representation

We wrote our own tokenizer instead of a standard word tokenizer because stylistic signals live in characters that most tokenizers throw away. The regular expression keeps words (case preserved, with contractions like *I'd* as one token), numbers, and *every* punctuation or markdown symbol as a separate token (`*`, `#`, `-`, `:`, ...). Line breaks become a special `<nl>` token, so a bulleted list looks like `<nl> - **` to the network. The vocabulary is built on the training split only (tokens with frequency ≥ 2, ~28,985 types); unseen tokens map to `<unk>`. Responses are truncated to 512 tokens (22% of responses are longer) and prompts to 128 tokens. The three input settings of RQ2 are: **output only**; **input only**; and **input + output** = prompt tokens, a `<sep>` token, then response tokens.

2.2 Models

CNN (Kim, 2014). A 128-d embedding layer (trained from scratch), then three parallel 1-D convolutions with kernel widths 3, 4 and 5 and 128 filters each (ReLU). Each feature map is max-pooled over time (padding positions are masked out), giving a 384-d vector that goes through dropout (0.5) and a linear layer to 6 logits. Each filter acts like a learned n-gram detector, e.g. `<nl> 1 . **` or *What a great question*, and max-pooling asks "does this pattern appear anywhere in the response?"

RNN: bidirectional LSTM. The same 128-d embedding (dropout 0.3), a single-layer bidirectional LSTM with 128 hidden units per direction, and a sentence vector made of the masked max-pool and the masked mean-pool of all hidden states (512-d), followed by dropout and a linear layer. We first used the final hidden states with `pack_padded_sequence`, but on our CPU-only setup a batch with mixed lengths took ~30 s versus ~1 s unpacked, so we run the LSTM on padded batches and rely on pooling with masks instead of the last state. Batches are bucketed by length so there is very little padding.

2.3 Training details

Table 2. Model and training hyper-parameters.

Setting	CNN	BiLSTM
Parameters	3.91 M	3.98 M
Embedding	128-d, random init, trained	128-d, random init, trained
Core layer	conv k=3,4,5 × 128 filters	BiLSTM 2 × 128 hidden
Pooling	masked max over time	masked max + mean over time
Dropout	0.5 (before output)	0.3 (embeddings and before output)
Optimizer	Adam, lr 1e-3, batch 64	Adam, lr 1e-3, batch 64
Gradient clipping	norm 5	norm 5
Epochs / early stop	max 15, patience 4 on val macro-F1	max 15, patience 4 on val macro-F1
Seeds (RQ1/RQ2)	3	2
Avg. time per output-only run	14 min (1 CPU thread)	18 min (1 CPU thread)

Loss is cross-entropy. After every epoch we evaluate macro-F1 on the validation prompts and keep the best checkpoint; training stops when validation F1 has not improved for 4 epochs. Test prompts are used only once, for the final numbers. We report the mean ± standard deviation over random seeds (3 for the CNN, 2 for the slower LSTM). Everything ran on a 2-core CPU without a GPU, which is why we kept the models small and the number of LSTM seeds low. Hyper-parameters were set once from common defaults and a single trial run, not tuned on the test set. As a non-neural reference point (not required) we also trained a TF-IDF (word 1–2-grams + character 2–5-grams) logistic-regression model on the same splits.

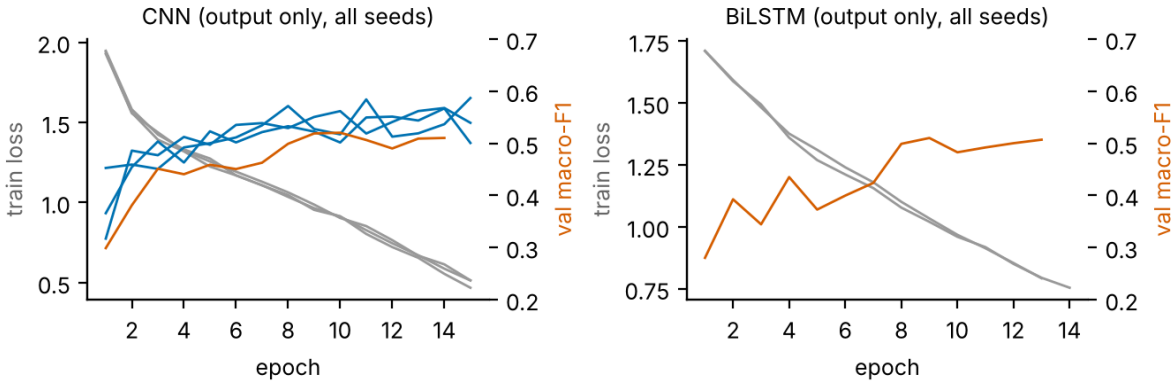


Figure 1. Training loss (grey) and validation macro-F1 per epoch, output-only setting, all seeds. Validation F1 is noisy because the validation set has only 120 prompts; early stopping keeps the best epoch.

3. Results (RQ1 and RQ2)

Table 3. Test results on 1,452 responses to 121 unseen prompts (6 balanced classes, chance = 16.7%). Mean ± std over seeds (CNN: 3, BiLSTM: 2).

Input used	CNN acc (%)	CNN macro-F1	BiLSTM acc (%)	BiLSTM macro-F1	TF-IDF+LR acc (reference)
Output only (RQ1)	56.2 ± 0.2	0.554	53.0 ± 0.9	0.518	60.5
Input only	16.7 ± 0.0	0.104	16.7 ± 0.0	0.105	16.7
Input + Output	56.6 ± 0.4	0.562	52.4 ± 1.4	0.504	59.4

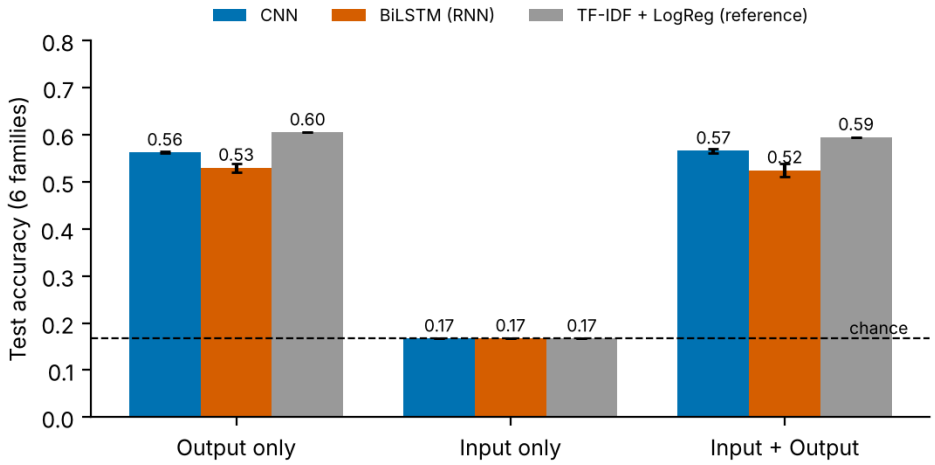


Figure 2. Test accuracy by input setting. Error bars: std over seeds. The dashed line is chance (1/6).

3.1 RQ1: Can we identify which LLM generated a response?

Yes, well above chance. From the response alone, the CNN identifies the family of an answer to a never-seen prompt 56.2% of the time (macro-F1 0.554), and the BiLSTM 53.0% (macro-F1 0.518), versus 16.7% for guessing. The CNN is 3.2 points better and much more stable across seeds (std 0.2 vs. 0.9). We think the CNN fits this task better because the evidence is local and position-independent (a bold heading, an opening phrase, a list marker), which is exactly what max-pooled n-gram filters detect, while the LSTM has to carry such evidence through hundreds of steps and is harder to optimize from scratch on ~6.7k examples. The TF-IDF reference reaches 60.5%, so a good part of the signal is available as surface n-grams; the neural models do not find much beyond it with this amount of data.

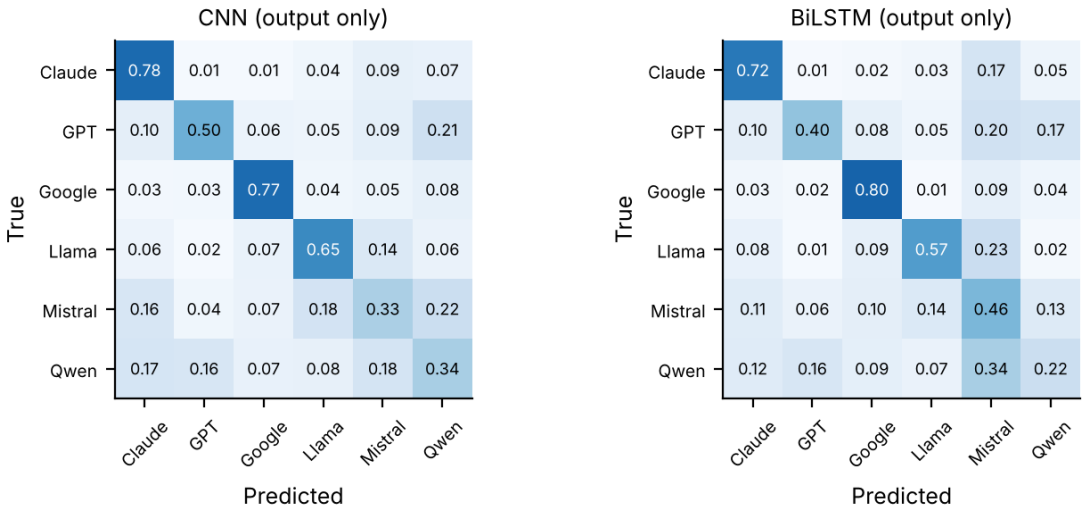


Figure 3. Row-normalized confusion matrices (output only, summed over seeds). Diagonal = recall per family.

Not all families are equally recognizable. CNN recall ranges from 77.7% (Claude) and 77.0% (Google) down to 34.0% (Qwen) and 33.5% (Mistral) (Figure 3). Google's models are distinctive because they often skip any introduction and start directly with a bold list item or give a one-line answer; Claude 3 almost never uses `**bold**` markdown in this data. Qwen and Mistral are the hardest: their answers look like a generic "average" assistant answer (an intro sentence, a numbered list, a short conclusion). They are most often confused with each other and with Claude, Mistral also with Llama and Qwen also with GPT. One plausible reason is that open models are often fine-tuned on conversations generated by, or modeled after, GPT-style assistants, which would blur their style.

Table 4. Per-model accuracy (CNN, output only, mean over seeds). The two members of a family are often not equally recognizable.

Family	Model	CNN acc on its responses (%)
Claude	claude-3-5-sonnet-20240620	87.1
Claude	claude-3-opus-20240229	68.3
GPT	gpt-4o-2024-05-13	61.4
GPT	gpt4_1106_preview	37.7
Google	gemini-pro	66.7
Google	gemma-7b-it	87.3
Llama	Meta-Llama-3-70B-Instruct	76.3
Llama	llama-2-70b-chat-hf	54.5
Mistral	Mixtral-8x7B-Instruct-v0.1	32.8
Mistral	mistral-large-2402	34.2
Qwen	Qwen1.5-72B-Chat	38.6
Qwen	Qwen2-72B-Instruct	29.5

Within families the gap can be large: e.g. claude-3.5-sonnet (87.1%) vs. claude-3-opus (68.3%), gpt-4o (61.4%) vs. gpt-4-turbo (37.7%), and Llama-3 (76.3%) vs. Llama-2 (54.5%). So a "family fingerprint" is partly the fingerprint of its most distinctive member, and successive versions of one vendor's model can drift apart in style.

3.2 RQ2: Does the user's prompt help identify the LLM?

(i) Can the LLM be predicted from the prompt alone? No, not in our data. The input-only CNN scores 16.7% and the BiLSTM 16.7%, i.e. chance. This is expected by construction: every prompt appears 12 times in the dataset, twice with each label, so the prompt carries zero information about the label. The networks correctly learn to output roughly the same distribution for every prompt. We consider this the most important sanity check of the dataset: if input-only had been above chance, our split or our data would have been leaking.

(ii) Does adding the input improve classification? Not meaningfully. Input + output gives 56.6% (CNN) and 52.4% (BiLSTM), compared with 56.2% and 53.0% for output only; TF-IDF also does not improve (59.4% vs. 60.5%). The changes are small and go in both directions, i.e. within run-to-run noise. In principle the prompt could help as *context* (e.g. knowing a one-line answer was given to a request for a long essay), but with our data size the models cannot learn such interactions, and the extra prompt tokens mostly add noise.

(iii) When can input-only classification work surprisingly well? Only when the dataset itself links prompts to models. That happens easily in practice: if one person collects GPT answers to coding questions and another collects Llama answers to chit-chat, a classifier can learn "coding prompt → GPT" without ever looking at the writing. To show this we built biased versions of our data: each prompt keeps a *single* response, and with probability p that response comes from the family we linked to the prompt's source collection (Self-Instruct→GPT, OpenAssistant→Claude, Koala→Llama, helpful-base→Qwen, Vicuna→Mistral), otherwise from a random other family. We trained the input-only CNN on each version (3 seeds).

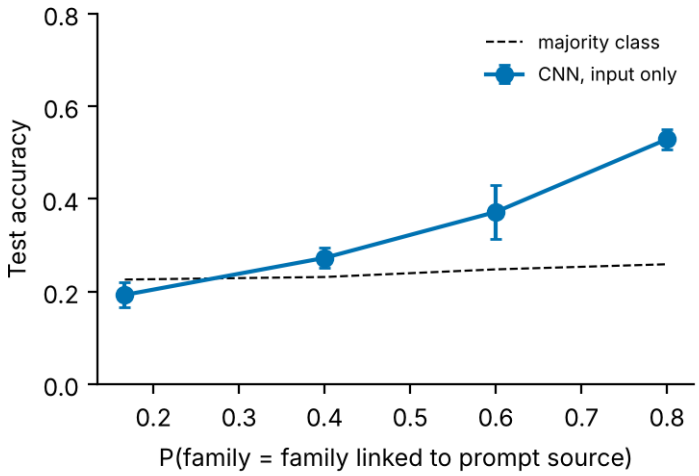


Figure 4. Input-only CNN accuracy when the data collection links prompt sources to families with probability p (p = 1/6: no link). Dashed: always predicting the most frequent class.

With no link (p = 1/6) the prompt-only model is at 19.3%, near chance. As the collection bias grows, the same model rises to 52.9% at p = 0.8 (majority-class baseline 25.9%) — without seeing a single word written by an LLM. The prompt sources differ in style (Self-Instruct prompts are templated tasks, Koala prompts are casual chat, etc.), so the network learns the source, and through the source the label. **So a high input-only score is a warning sign of dataset bias, not evidence of an LLM fingerprint.** This is why we used a dataset where every family answers every prompt.

5. Lessons and Experience

The split matters more than the model. A random row split is the default in most tutorials. With 12 answers per prompt, it would have put most test prompts into training under other labels and let topic words leak. Splitting by prompt was a small code change but it decides what the reported number means. The input-only result (exactly chance) is the sanity check that the split works.

Dataset design answers research questions before any training. Choosing a dataset where every family answers the same prompts made RQ2 almost trivially answerable for our data, and the bias simulation showed us how easily a less controlled dataset produces an impressive but meaningless input-only score. When we read about "AI text detectors" with very high accuracy, the first question we now ask is how the data were collected.

Tokenization is a modeling decision. A standard tokenizer would have deleted most of the signal: markdown symbols, list markers and line breaks are exactly where the families differ most visibly (e.g. Claude 3 almost never writes **bold**, Gemini often starts with a bulleted list).

A simple baseline keeps you honest. The TF-IDF + logistic regression model trained in about a minute and was at least as accurate as both neural networks. With only ~6,700 training examples and no pre-trained embeddings, the CNN and LSTM must learn word meanings from scratch, while n-gram counts capture the surface habits directly. A pre-trained transformer would likely do better; we stayed with the required architectures and a CPU budget.

CNN vs. RNN in practice. The CNN was faster (≈57 s vs. ≈84 s per epoch on one CPU thread), more stable across seeds and more accurate. Our LSTM was also slowed by an engineering problem: packed sequences were ~30× slower on our CPU build, and we had to switch to padded, length-bucketed batches with masked pooling. Profiling one batch before launching hours of training would have saved us a failed run.

Limitations. (i) All responses come from one benchmark with default settings and no system prompts; real users change temperature, system prompts and formatting instructions, which would weaken formatting cues. (ii) Two models per family is the minimum to talk about a "family"; newer versions may write differently (e.g. Claude 3.5 Sonnet already differs from Claude 3 Opus in our per-model results). (iii) Because of the CPU budget we ran 3 seeds for the CNN and 2 for the BiLSTM, and did no hyper-parameter search.

References

Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. EMNLP.

Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.

Li, X., Zhang, T., Dubois, Y., et al. (2023). AlpacaEval: An Automatic Evaluator of Instruction-following Models. github.com/tatsu-lab/alpaca_eval.

Monroe, B., Colaresi, M., & Quinn, K. (2008). Fightin' Words: Lexical Feature Selection and Evaluation for Identifying the Content of Political Conflict. Political Analysis.

Covington, M. & McFall, J. (2010). Cutting the Gordian Knot: The Moving-Average Type-Token Ratio (MATTR). Journal of Quantitative Linguistics.